Building a Lightweight, Auditable Personal AI Assistant with HITL

- Building a Lightweight, Auditable Personal AI Assistant with HITL
  - Deep Research Report — March 2026
  - 1. Executive Summary
  - 2. Area 1: Survey of Existing Personal AI Assistants
    - 2.1 Comparison Table
    - 2.2 Detailed Analysis of Key Frameworks
  - 3. Area 2: Big Challenges for Introducing a New Assistant
    - 3.1 State Management and Memory Persistence
    - 3.2 Multi-Model Orchestration
    - 3.3 Security and Sandboxing
    - 3.4 Audit Trails and Compliance
    - 3.5 Human-in-the-Loop Patterns
    - 3.6 Context Window Management
    - 3.7 Tool/Integration Ecosystem (MCP Adoption)
    - 3.8 Cost Management
    - 3.9 Reliability and Error Recovery
    - 3.10 Privacy and Data Sovereignty
    - 3.11 Real-Time vs Async Execution
  - 4. Area 3: Native Integrations and Composability
    - 4.1 Integration Priority Matrix
    - 4.2 Key Architecture Decisions
  - 5. Area 4: Enterprise-Grade Architecture with HITL
    - 5.1 Durable Execution Frameworks
    - 5.2 Ruflo — Agent Orchestration Platform
    - 5.3 Enterprise HITL Patterns
    - 5.4 Audit Logging Standards
    - 5.5 Role-Based Access Control
    - 5.6 Cost Attribution and Chargeback
  - 6. Architecture Proposal
    - 6.1 High-Level Architecture
    - 6.2 Key Design Principles
  - 7. Recommended Tech Stack
  - 8. MVP Roadmap
    - Phase 1: Foundation (Weeks 1-4)
    - Phase 2: Channels & Tools (Weeks 5-8)
    - Phase 3: Desktop & Polish (Weeks 9-12)
    - Phase 4: Mobile & Enterprise (Weeks 13-20)
  - 9. Sources
    - Frameworks & Tools
    - Orchestration & Infrastructure
    - Research & Guides

Building a Lightweight, Auditable Personal AI Assistant with HITL

Deep Research Report — March 2026

---

1. Executive Summary

The personal AI assistant landscape has exploded since late 2025. OpenClaw (247K+ GitHub stars) proved the market wants **local-first, agentic runtimes** that do things rather than just chat. But the field has critical gaps: audit trails are an afterthought, HITL (human-in-the-loop) is bolted on rather than native, security is alarming (Cisco found data exfiltration in OpenClaw skills), and cost management is opaque.

Our opportunity: Build a lightweight, auditable personal AI assistant where HITL is a first-class primitive, not a feature toggle. The architecture should be: - **Durable** (Temporal-style execution that survives crashes) - **Auditable** (immutable, structured logs from day one) - **Composable** (MCP-native for tool integration) - **Multi-model** (intelligent routing with cost controls) - **Human-first** (approval queues, escalation matrices, trust tiers)

This report surveys 19 existing frameworks, identifies the 11 hardest technical challenges, maps the integration ecosystem, and proposes an enterprise-grade architecture built on durable execution with native HITL.

---

2. Area 1: Survey of Existing Personal AI Assistants

2.1 Comparison Table

Framework	Type	Architecture	Pricing	HITL Support	Audit Trail	MCP Support	Open Source
OpenClaw	Agentic Runtime	TypeScript/ Swift gateway + LLM + messaging channels	Free (OSS) + LLM costs	Basic (chat-based approval)	Local session logs, no structured audit	Yes (client)	MIT
Open Interpreter	Local Code Exec	Python CLI → LLM → local code execution	Free (OSS) + LLM costs	Manual confirmation per command	Terminal history only	No	AGPL-3.0
Aider	AI Pair Programming	Python CLI, git-aware, repo map, diff-based edits	Free (OSS) + LLM costs (~$0.007/ file)	Git commit approval	Git history as audit trail	No	Apache 2.0
AutoGPT/ AgentGPT	Autonomous Agents	Python, task decomposition loops, plugin system	Free (OSS) + LLM costs	Minimal—designed for autonomy	Basic logging	No	MIT

Framework	Type	Architecture	Pricing	HITL Support	Audit Trail	MCP Support	Open Source
CrewAI	Multi-Agent Orchestration	Python, role/task abstractions, crew collaboration	Free (OSS) + Enterprise pricing	Built-in HITL task callbacks	Crew execution logs	No	MIT
LangGraph	Agent Framework	Python/JS, graph-based state machines, checkpointing	Free (OSS) + LangSmith ($39+/mo)	Native interrupt/resume nodes	LangSmith traces	No (uses tools)	MIT
Dust.tt	Enterprise AI Assistants	Cloud SaaS, RAG-first, workspace knowledge	€29/user/mo Pro; Custom Enterprise	Admin approval workflows	Enterprise audit logs	No	Proprietary
Lindy.ai	Personal AI Assistant	Cloud SaaS, no-code agent builder, 5K+ integrations	$49.99/mo+ (credit-based)	Escalation to humans	SOC2/HIPAA compliant	No	Proprietary
Relevance AI	AI Workforce	Cloud SaaS, no-code agent builder	Free tier; Pro $19/mo; Team $199/mo	Task review queues	Enterprise tier only	No	Proprietary
n8n AI Agents	Workflow Automation	Self-host or cloud, visual workflow builder, 500+ connectors	Free (OSS); Cloud €20-667/mo	Native workflow pauses	Run history, error logs	No	Fair-code
Taskade AI	AI Project Management	Cloud SaaS, workspace/project-based	$8-16/mo per user	Task assignment/review	Basic activity logs	No	Proprietary
Claude MCP	Protocol/Ecosystem	JSON-RPC 2.0, stdio/HTTP transport, tool discovery	Free (protocol); tools vary	Tool-level permissions	Transport-level logging	**IS** MCP	Apache 2.0
Microsoft AutoGen	Multi-Agent Framework	Python/.NET, event-driven, async message passing	Free (OSS)	Native HITL agent type	Azure telemetry integration	No	MIT
GPT Pilot	AI Developer	Python, step-by-step app generation, clarifying dialogue	Free (OSS) + LLM costs	Built-in—asks clarifying questions	Development log files	No	MIT
Devin	AI Software Engineer	Cloud SaaS, autonomous IDE/terminal/browser	$20/mo+ ($2.25/ACU)	PR review cycle as HITL	Full session recordings	No	Proprietary
					Repl history	No	Proprietary

Framework	Type	Architecture	Pricing	HITL Support	Audit Trail	MCP Support	Open Source
Replit Agent	AI Coding Agent	Cloud IDE, Agent 3, full-stack app generation	$20-59/mo per user	Chat-based guidance			
Cursor	AI IDE	VS Code fork, inline AI, multi-file edits	$20/seat/mo Pro	Accept/reject diffs	Git integration	MCP client	Proprietary
Windsurf	AI IDE	VS Code fork, Cascade flow, agentic mode	$15/seat/mo Pro	Accept/reject diffs	Git integration	MCP client	Proprietary
Composio	Tool Integration Layer	Python/JS SDK, 250+ integrations, OAuth management	Free tier; paid plans	Delegated auth approval	Integration logs	Full MCP support	Apache 2.0
Arcade AI	Tool Use Platform	Tool SDK + Engine + Actor system, OAuth-native	Free tier; Enterprise custom	Auth-gated tool execution	Execution audit logs	MCP server support	MIT

2.2 Detailed Analysis of Key Frameworks

OpenClaw (What We Use)

- **Architecture:** TypeScript gateway process running locally. Connects to LLMs (Claude, GPT, DeepSeek) via API. User interface through messaging platforms (Discord, Signal, Telegram, WhatsApp). Skills (plugins) extend functionality. Configuration and history stored locally in `.clawdbot/` directory.
- **Strengths:** Incredibly flexible; open-source with massive community (247K stars); multi-channel; local-first data; skill ecosystem via ClawHub; MCP client support; proactive automation via cron.
- **Weaknesses:** Security concerns (Cisco found malicious skills); prompt injection susceptible; no structured audit trail; HITL is chat-based only (no formal approval queues); skill vetting is weak; complexity too high for casual users; no native mobile app.
- **Key Insight:** OpenClaw proved the market but isn't enterprise-ready. Its creator (Peter Steinberger) left for OpenAI in Feb 2026, and the project moved to an open-source foundation. Future governance uncertain.
- **Source:** https://en.wikipedia.org/wiki/OpenClaw, https://github.com/openclaw/openclaw

LangGraph (LangChain)

- **Architecture:** Graph-based state machine abstraction over LangChain. Nodes represent agent steps, edges represent transitions. Supports checkpointing for state persistence, explicit branching, and multi-agent orchestration.
- **Strengths:** Explicit state control; checkpoint/resume; huge ecosystem; Python and JS; composable with LangSmith for observability; supports HITL via interrupt nodes.
- **Weaknesses:** Steep learning curve; doc sprawl; bloated imports from LangChain layers; no visual builder; deployment and governance are DIY.

- **HITL:** First-class. Can define interrupt nodes that pause graph execution until human input is provided.
- **Source:** https://www.langchain.com/langgraph, https://langwatch.ai/blog/best-ai-agent-frameworks-in-2025

CrewAI

- **Architecture:** Role-based multi-agent. Define Agents with roles, Tasks with descriptions, and Crews that orchestrate execution. Supports sequential and hierarchical process flows.
- **Strengths:** Clean abstractions; natural metaphor for dividing work; growing enterprise features; broad LLM support.
- **Weaknesses:** Python-only; can be overkill for simple flows; maturity varies; less focused on visual/no-code.
- **HITL:** Supports `human_input=True` on tasks for manual intervention points.
- **Source:** https://crewai.com, https://www.langflow.org/blog/the-complete-guide-to-choosing-an-ai-agent-framework-in-2025

Microsoft AutoGen / Agent Framework

- **Architecture:** Event-driven, async message passing between agents. v0.4 introduced more robust architecture. Now converging with Semantic Kernel into unified "Microsoft Agent Framework" with session-based state management, type safety, middleware, telemetry, and graph-based workflows.
- **Strengths:** Enterprise backing (Microsoft); Python + .NET; Azure integration; AutoGen Studio (no-code GUI); multi-agent conversation patterns; native HITL agent type.
- **Weaknesses:** Tight Azure coupling for production; rapid API changes; convergence with Semantic Kernel still in progress.
- **Source:** https://github.com/microsoft/autogen, https://learn.microsoft.com/en-us/agent-framework/overview

Claude MCP (Model Context Protocol)

- **Architecture:** Not a framework but a **protocol standard**. JSON-RPC 2.0 transport. Defines how LLMs discover and call tools via MCP servers. Donated to Agentic AI Foundation (Linux Foundation) in Dec 2025. Adopted by OpenAI, Anthropic, Google, Microsoft.
- **Strengths:** Industry standard for tool integration; transport-agnostic (stdio, SSE, HTTP streamable); massive ecosystem of servers; endorsed by all major LLM providers.
- **Weaknesses:** Security challenges (Knostic found all 2,000 scanned MCP servers lacked authentication in July 2025); still evolving; no built-in auth standard.
- **Key Insight:** MCP is the **USB-C of AI tools**. Any new assistant MUST be MCP-native.
- **Source:** https://en.wikipedia.org/wiki/Model_Context_Protocol, https://www.thoughtworks.com/en-us/insights/blog/generative-ai/model-context-protocol-mcp-impact-2025

Composio

- **Architecture:** Tool integration layer. 250+ pre-built integrations with managed OAuth. Works with 25+ agent frameworks. Supports MCP as both client and server.
- **Strengths:** Solves the hardest part (auth + integrations); framework-agnostic; MCP-native; handles OAuth token management.
- **Weaknesses:** Adds dependency layer; cloud-managed auth introduces trust considerations; pricing can scale.

- **Key Insight:** Rather than building integrations ourselves, Composio (or Arcade) can serve as the integration backbone. Build on top, not from scratch.
- **Source:** https://composio.dev, https://mcp.composio.dev

Devin (Cognition Labs)

- **Architecture:** Cloud-hosted autonomous software engineer. Has its own IDE, terminal, and browser. Works asynchronously on tasks assigned via Slack or web interface.
- **Strengths:** Most autonomous coding agent; handles full development cycles; Devin 2.0 price dropped to $20/mo; 12x efficiency improvement reported for migrations.
- **Weaknesses:** Proprietary black box; cloud-only; expensive at scale ($2.25/ACU); limited to coding tasks; no self-hosting.
- **Source:** https://devin.ai, https://venturebeat.com/programming-development/devin-2-0-is-here

Lindy.ai

- **Architecture:** Cloud SaaS no-code platform. Agent Builder (natural language), Lindy Build (app generation with auto-testing), Computer Use (browser automation), Gaia (voice agent).
- **Strengths:** 5,000+ integrations; SOC2/HIPAA/GDPR compliant; Claude Sonnet 4.5 integration; 30+ hours autonomous operation; Computer Use for non-API workflows.
- **Weaknesses:** Premium pricing ($49.99/mo+); credit-based; not self-hostable; less suitable for deterministic workflows.
- **Pricing:** Free (400 credits) → Premium ($49.99/mo) → Business ($199.99/mo)
- **Source:** https://www.lindy.ai, https://max-productive.ai/ai-tools/lindy/

---

3. Area 2: Big Challenges for Introducing a New Assistant

3.1 State Management and Memory Persistence

The Problem: LLMs are stateless. Every session starts from zero. Building persistent memory that's useful, not bloated, is the #1 unsolved problem.

Current Approaches: | Approach | Used By | Pros | Cons | |———-|———|——|——| | File-based (markdown) | OpenClaw | Simple, human-readable, version-controlled | Doesn't scale; no semantic search; manual curation needed | | Vector DB (embeddings) | LangChain, Agno | Semantic search; scales well | Embedding drift; no structured reasoning; retrieval quality varies | | Graph memory | Neo4j-based agents | Relationship-aware; structured | Complex setup; query overhead | | Hybrid (BM25 + vectors + rerank) | QMD (our current setup) | Best retrieval quality | Multiple services to maintain | | Modular memory (general + user-specific) | Research frontier | Clean separation; reduces over-personalization | Implementation complexity |

Recommendation: Adopt a **tiered memory architecture**: 1. **Working memory** — Current session context (in-context) 2. **Episodic memory** — Structured session logs with timestamps (SQLite + FTS5) 3. **Semantic memory** — Embeddings for knowledge retrieval (local vector DB) 4. **Procedural memory** — Learned workflows and preferences (versioned files) - Source: https://arxiv.org/abs/2512.13564

3.2 Multi-Model Orchestration

The Problem: No single model is best at everything. GPT-5 excels at reasoning, Claude at code, DeepSeek at cost efficiency. Routing intelligently saves 30-60% on costs.

Solutions Emerging: - **LiteLLM** — Unified API for 100+ models with fallback/retry - **Portkey AI** — Enterprise AI gateway with routing, caching, guardrails - **RouteLLM** — Learned routing between strong/weak models (research) - **Pick and Spin** — 21.6% higher success rates, 30% lower latency, 33% lower GPU cost (Dec 2025 paper)

Recommendation: Use **LiteLLM as the base gateway** with custom routing rules. Start simple (task-type → model mapping), evolve to learned routing.

3.3 Security and Sandboxing

The Problem: Autonomous agents that can execute code, access files, and call APIs are inherently dangerous. OpenClaw's MoltMatch incident showed agents acting beyond user intent.

Threats: - Prompt injection via untrusted data (emails, web content, skills) - Data exfiltration through malicious tools/skills - Privilege escalation via tool chains - Supply chain attacks through skill/plugin ecosystems

Solutions: - **Kubernetes Agent Sandbox** (Google, Nov 2025) — Formal K8s subproject for agent execution sandboxing - **Docker/gVisor** — Container-based isolation for code execution - **Seccomp profiles** — Syscall filtering for sandboxed processes - **Capability-based permissions** — Tools declare required permissions; user approves

Recommendation: Every tool execution should run in a **sandboxed subprocess** with declared permissions. High-risk actions (file deletion, network access, financial transactions) require explicit HITL approval. Use Docker for code execution sandboxing.

3.4 Audit Trails and Compliance

The Problem: For enterprise adoption, every agent action needs to be traceable. SOC2 requires demonstrating control over automated processes. HIPAA requires audit logs for health data access.

Standards: - **Structured logging** — JSON-structured events with timestamps, action types, inputs/outputs - **Immutable audit trails** — Append-only logs (SQLite WAL mode or dedicated audit DB) - **OpenTelemetry** — Industry standard for distributed tracing - **Event sourcing** — Store every state change as an event; reconstruct state at any point

Recommendation: Build audit logging as a **core primitive, not a feature**:

```
{
  "timestamp": "2026-03-03T10:00:00Z",
  "session_id": "abc-123",
  "action": "tool_call",
  "tool": "send_email",
  "input": {"to": "...", "subject": "..."},
  "output": {"status": "sent"},
  "approval": {"required": true, "approver": "user:nag",
"approved_at": "..."},
  "cost": {"tokens_in": 1200, "tokens_out": 300, "cost_usd":
```

```
0.012},
  "model": "claude-sonnet-4.5",
  "duration_ms": 2300
}
```


3.5 Human-in-the-Loop Patterns

The Problem: Pure autonomy is dangerous. Pure manual approval is slow. The sweet spot is **graduated autonomy** based on risk and trust.

Patterns Identified:

Pattern	Description	When to Use
Pre-execution approval	Agent proposes action, human approves	High-risk (send money, delete data, public posts)
Post-execution review	Agent acts, human reviews after	Medium-risk (draft emails, code changes)
Escalation	Agent tries, escalates on failure/uncertainty	Edge cases, out-of-scope requests
Confidence-based	Auto-approve if confidence > threshold	Routine tasks with clear success criteria
Budget-based	Auto-approve if cost < threshold	Cost-controlled operations
Time-boxed	Auto-approve if no human response within N minutes	Non-critical, time-sensitive tasks

Recommendation: Implement a **policy engine** that maps (action_type, risk_level, trust_tier, cost) → approval_requirement. Users configure their comfort level. Default to conservative.

Source: https://zapier.com/blog/human-in-the-loop/, https://www.permit.io/blog/human-in-the-loop-for-ai-agents-best-practices, https://learn.microsoft.com/en-us/agent-framework/workflows/human-in-the-loop

3.6 Context Window Management

Challenge: Even with 200K+ context windows, real-world agents hit limits fast when loading memory, tool outputs, and conversation history.

Strategies: - **Progressive disclosure** — Load minimal context, fetch more on demand (claude-mem pattern) - **Summarization chains** — Compress old messages into summaries - **RAG over history** — Retrieve relevant past context instead of loading all - **Tiered context** — System prompt (always) → relevant memory (retrieved) → recent conversation (sliding window)

3.7 Tool/Integration Ecosystem (MCP Adoption)

Status (March 2026): - MCP donated to Agentic AI Foundation (Linux Foundation, Dec 2025) - Co-founded by Anthropic, Block, and OpenAI - OpenAI adopted MCP across Agents SDK, Responses API, and ChatGPT Desktop (March 2025) - HTTP Streamable transport is the recommended standard in 2026 - Security remains the biggest concern: nearly 2,000 scanned MCP servers lacked authentication

Recommendation: Build as an **MCP-first architecture**. All tools are MCP servers. All integrations go through MCP. This future-proofs against ecosystem changes.

3.8 Cost Management

Reality Check: - GPT-4o: ~$2.50/1M input tokens, ~$10/1M output tokens - Claude Sonnet 4: ~$3/1M input, ~$15/1M output - DeepSeek V3: ~$0.27/1M input, ~$1.10/1M output - A single complex agent task can cost $0.10-$5.00

Strategies: - Task-based model routing (use cheap models for classification, expensive for reasoning) - Aggressive caching (semantic similarity cache for repeated queries) - Token budgets per task/user/day - Cost attribution in audit logs (know what costs what)

3.9 Reliability and Error Recovery

Core Challenge: LLMs are probabilistic. They hallucinate, fail mid-task, return malformed outputs.

Solutions: - **Durable execution** (Temporal/Inngest) — Automatic retry with state persistence - **Structured output** (JSON schemas, Pydantic models) — Validate LLM outputs - **Fallback chains** — If primary model fails, try secondary - **Circuit breakers** — Stop cascading failures in multi-step workflows

3.10 Privacy and Data Sovereignty

Considerations: - Local-first data storage (no cloud dependency for core data) - Configurable data sharing (opt-in per integration) - Encryption at rest for sensitive data (SQLite + SQLCipher) - Clear data boundaries (what leaves the machine, what stays)

3.11 Real-Time vs Async Execution

Both are needed: - **Real-time:** Chat responses, quick tool calls, interactive coding - **Async:** Long-running research, batch operations, scheduled tasks, approval-gated workflows

Architecture implication: Need both a request/response path AND a durable task queue.

---

4. Area 3: Native Integrations and Composability

4.1 Integration Priority Matrix

Integration	Impact	Effort	Priority	Approach
MCP Server Ecosystem	🔥🔥🔥	Low	**P0**	Native MCP client/server
Discord/Slack/Teams	🔥🔥🔥	Medium	**P0**	Messaging channel adapters
Calendar/Email (Google/M365)	🔥🔥🔥	Medium	**P0**	Via MCP servers or Composio
CLI/Terminal	🔥🔥🔥	Low	**P0**	Native (core interface)
Database (SQLite/Postgres)	🔥🔥	Low	**P0**	Native for state; MCP for user DBs
File System Watchers	🔥🔥	Low	**P1**	Native (inotify/FSEvents)

Integration	Impact	Effort	Priority	Approach
Webhook/API Gateway	🔥🔥🔥	Medium	**P1**	HTTP server for external triggers
IDE Plugins (VS Code)	🔥🔥	Medium	**P1**	VS Code extension + MCP
Browser Extension	🔥🔥	Medium	**P1**	Chrome extension for web context
Mobile App (iOS/ Android)	🔥🔥🔥	High	**P1**	Tauri 2.0 (shared core) or React Native
Desktop App	🔥🔥	Medium	**P1**	Tauri 2.0 (Rust core + web frontend)
Smart Home (HomeKit/HA)	🔥	Medium	**P2**	Home Assistant MCP server
Voice (Siri/Alexa)	🔥🔥	High	**P2**	Siri Shortcuts + webhook bridge
CI/CD Pipelines	🔥🔥	Medium	**P2**	GitHub Actions integration
Observability (OTel/ Grafana)	🔥🔥	Medium	**P2**	OpenTelemetry SDK built-in
Zapier/Make	🔥	Low	**P2**	Webhook-based
Wearables (Apple Watch)	🔥	High	**P3**	Push notifications via mobile app
Redis	🔥	Low	**P3**	MCP server

4.2 Key Architecture Decisions

Tauri 2.0 for Desktop + Mobile: - Same Rust core, same web frontend, native mobile packaging - 60-80% smaller than Electron - System-level access (file system, notifications, tray) - Stable since 2025-2026, production-grade - Source: https://v2.tauri.app

MCP as the Universal Integration Layer: - All external tool integrations should be MCP servers - Core assistant is both MCP client (consuming tools) and MCP server (exposable to other agents) - Use Composio or Arcade for pre-built OAuth-managed integrations - Build custom MCP servers for proprietary systems

Event-Driven Architecture: - Webhooks and file system events trigger workflows - Internal event bus for agent-to-agent communication - External events (Slack message, email arrival, CI failure) all normalize to events

---

5. Area 4: Enterprise-Grade Architecture with HITL

5.1 Durable Execution Frameworks

Temporal.io ⭐ Recommended

- **What:** Durable execution platform. Workflows survive crashes, retain state indefinitely, support human-in-the-loop natively.

- **Architecture:** Temporal Server (orchestrator) + Workers (execute activities). Event History is append-only log of every state change.
- **For AI Agents:** Built-in signal/query for HITL; workflows can pause for human input; automatic retry for LLM failures; multi-language (Go, Python, Java, TypeScript, .NET).
- **HITL Support:** Native. Signal workflows to pause, collect human input, resume. Temporal Cloud: $1K free credits for new users.
- **Pricing:** Open source (self-host) or Temporal Cloud (usage-based, ~$200/mo starting).
- **Source:** https://temporal.io, https://temporal.io/blog/build-resilient-agentic-ai-with-temporal

Inngest

- **What:** Event-driven serverless workflow platform. Designed for AI orchestration.
- **Architecture:** Functions triggered by events. Steps within functions are individually retried. Built-in `step.ai` for LLM calls with observability.
- **Strengths:** Serverless-first; great DX (TypeScript-native); `useAgent` React hook for streaming; runs anywhere (edge, serverless, traditional).
- **HITL:** Via `step.waitForEvent()` — pause function until human-triggered event.
- **Pricing:** Free tier → Pro $50/mo → Enterprise custom.
- **Source:** https://www.inngest.com, https://www.inngest.com/blog/ai-orchestration-with-agentkit-step-ai

Hatchet

- **What:** Distributed, fault-tolerant task queue. Purpose-built for AI agent workflows.
- **Architecture:** Worker-based execution. Tasks defined as simple functions. Built-in eventing for HITL signaling and streaming responses.
- **Strengths:** Designed for long-running AI tasks; HITL as first-class; self-hostable; simpler than Temporal.
- **Source:** https://hatchet.run, https://github.com/hatchet-dev/hatchet

Comparison Table

Feature	Temporal	Inngest	Hatchet	Prefect	Dagster
Primary Use	Durable workflows	Event-driven functions	Background tasks	Data pipelines	Data assets
HITL Native	✅ Signals/queries	✅ waitForEvent	✅ Event-based	⚠️ Manual	⚠️ Manual
Self-Hostable	✅	✅	✅	✅	✅
AI-Optimized	✅ (growing)	✅ (step.ai)	✅ (built for it)	❌	❌
Language	Multi (Go, Python, TS, Java)	TypeScript-first	Go/Python/TS	Python	Python
Audit Trail	✅ Event History (immutable)	✅ Function run logs	✅ Task logs	✅ Flow runs	✅ Asset logs
Complexity	High	Medium	Medium	Medium	Medium
Best For	Complex, long-running workflows	Serverless AI orchestration	AI agent task queues	Data pipelines	Data-centric pipelines

5.2 Ruflo — Agent Orchestration Platform

What we found: Ruflo (github.com/ruvnet/ruflo) is an agent orchestration platform for Claude. It features: - Multi-agent swarm deployment - Distributed swarm intelligence - RAG integration - Native Claude Code / Codex integration - Enterprise-grade architecture claims - 444 open issues, 97 open PRs (active development)

Assessment: Interesting for multi-agent patterns but tightly coupled to Claude. Not a general-purpose work order system. Better as reference architecture than direct dependency.

5.3 Enterprise HITL Patterns

Approval Queues

```
Task Created → Risk Assessment → Route to Queue:
  - Low risk → Auto-approve (log only)
  - Medium risk → Notify human, auto-approve after timeout
  - High risk → Block until explicit approval
  - Critical → Require multi-approver consensus
```

Escalation Matrices

Level	Trigger	Responder	SLA
L1	Agent uncertain	Primary user	5 min
L2	L1 timeout or rejection	Designated backup	15 min
L3	Security/compliance flag	Admin/owner	1 hour
L4	System failure	On-call engineer	30 min

Work Order Pattern

```
TICKET → TRIAGE → ASSIGN → EXECUTE → REVIEW → CLOSE
  ↑          ↓          ↓          ↓          ↓
  └── REJECT ←── REJECT ←── FAIL ←── REJECT
```

Each state transition is an audit event. Agent can execute; human reviews. Rejection sends back with feedback.

5.4 Audit Logging Standards

Recommended Stack: - **Storage:** SQLite with WAL mode (local) or Postgres (multi-user) - **Format:** Structured JSON events (OpenTelemetry-compatible) - **Immutability:** Append-only table with hash chains (each event includes hash of previous) - **Retention:** Configurable per compliance level (SOC2: 1 year, HIPAA: 6 years) - **Export:** Standard formats (JSONL, CSV) for compliance auditors

5.5 Role-Based Access Control

Role	Permissions	Example
Owner	Full access, configure policies, manage users	Primary user

Role	Permissions	Example
Admin	Manage tools, review audit logs, approve high-risk	IT admin
Operator	Execute tasks, approve medium-risk	Team member
Viewer	Read-only access to outputs and logs	Auditor
Agent	Execute within policy bounds, escalate when uncertain	AI assistant

5.6 Cost Attribution and Chargeback

Every agent action logs: - Token count (input/output) - Model used and per-token cost - Tool execution costs (API calls, compute time) - Attribution to user/project/task

Enable per-user/per-project budgets with alerts at 80%/100% thresholds.

---

6. Architecture Proposal

6.1 High-Level Architecture

```
┌──────────────────────────────────────────────────────────────────────┐
│                              USER INTERFACES                         │
│  CLI │ Discord │ Slack │ Mobile (Tauri) │ Web │ API                  │
└──────────────────────────────────────────────────────────────────────┘
                                  │
                                  ▼
┌──────────────────────────────────────────────────────────────────────┐
│                              API GATEWAY                             │
│  Auth │ Rate Limiting │ Request Routing │ WebSocket                  │
└──────────────────────────────────────────────────────────────────────┘
                                  │
                                  ▼
┌──────────────────────────────────────────────────────────────────────┐
│                          ORCHESTRATION LAYER                         │
│                                                                      │
│   ┌──────────────┐   ┌──────────────┐   ┌──────────────────────┐    │
│   │ Policy       │   │ Approval     │   │ Task Router          │    │
│   │ Engine       │   │ Queue        │   │ (risk assessment)    │    │
│   └──────────────┘   └──────────────┘   └──────────────────────┘    │
│          │                  │                     │                  │
│          ▼                  ▼                     ▼                  │
│   ┌──────────────────────────────────────────────────────────┐       │
│   │                DURABLE EXECUTION ENGINE                  │       │
│   │           (Temporal / Inngest / Hatchet)                 │       │
│   │           - Workflow state persistence                   │       │
│   │           - HITL signal/resume                           │       │
│   │           - Automatic retry & recovery                   │       │
│   └──────────────────────────────────────────────────────────┘       │
└──────────────────────────────────────────────────────────────────────┘
                                  │
                                  ▼
┌──────────────────────────────────────────────────────────────────────┐
│                              AGENT CORE                              │
│                                                                      │
│   ┌──────────────┐   ┌──────────────┐   ┌──────────────────────┐    │
│   │ LLM          │   │ Memory       │   │ Context              │    │
```

```
│  Router        Manager        Assembler                  │
│  (LiteLLM)     (tiered)       (progressive)              │
└──────────────────────────────────────────────────────────┘
                          │
                          ▼
┌──────────────────────────────────────────────────────────┐
│                    TOOL LAYER (MCP)                      │
│                                                          │
│  Built-in      Composio       Custom MCP                 │
│  MCP           (OAuth         Servers                    │
│  Servers        mgmt)                                    │
└──────────────────────────────────────────────────────────┘
                          │
                          ▼
┌──────────────────────────────────────────────────────────┐
│                    DATA LAYER                            │
│                                                          │
│  SQLite        Vector         File System                │
│  (state,       Store          (workspace,                │
│   audit)       (memory)        configs)                  │
└──────────────────────────────────────────────────────────┘
```

6.2 Key Design Principles

1. **Audit-first:** Every action produces an audit event before executing
2. **HITL-native:** Approval requirements computed by policy engine, not hardcoded
3. **MCP-everything:** All tools are MCP servers; the assistant is both client and server
4. **Local-first:** Data stays on-device by default; cloud sync is opt-in
5. **Durable:** Long-running tasks survive crashes via execution engine
6. **Multi-model:** LLM router selects best model per task; user configures preferences
7. **Composable:** Every component is replaceable (swap Temporal for Inngest, swap SQLite for Postgres)

---

7. Recommended Tech Stack

Layer	Technology	Rationale
Runtime	TypeScript (Bun)	Fast startup, great ecosystem, async-native, OpenClaw precedent
Desktop/Mobile	Tauri 2.0	Rust core + web frontend, cross-platform including mobile, tiny binaries
Durable Execution	Temporal (primary) or Hatchet (simpler alternative)	Best HITL support, multi-language, proven at scale
LLM Gateway	LiteLLM	100+ model support, fallback, caching, cost tracking
Database	SQLite (single-user) / Postgres (multi-user)	Local-first, embedded, excellent for audit logs

Layer	Technology	Rationale
Vector Store	SQLite-vec or LanceDB	Embedded, no separate service, good enough for personal use
MCP SDK	Official MCP TypeScript SDK	Standard library for MCP client/server
Tool Integration	Composio (managed) + Custom MCP servers	Pre-built OAuth + custom tools
Observability	OpenTelemetry → local Grafana or stdout	Industry standard, exportable
Auth	Passkeys + JWT	Modern, passwordless, secure
Messaging	Adaptors (Discord.js, Slack Bolt, etc.)	Channel-agnostic message interface
Search	SQLite FTS5 + embedded vectors	No external services needed

---

8. MVP Roadmap

Phase 1: Foundation (Weeks 1-4)

Goal: Core loop working — user sends message, agent thinks, proposes action, user approves, agent executes.

- [ ] Project scaffolding (TypeScript/Bun, SQLite, basic CLI)
- [ ] LLM integration via LiteLLM (Claude + GPT + DeepSeek)
- [ ] Structured audit logging (every action → SQLite)
- [ ] Basic HITL: all tool calls require approval via CLI
- [ ] MCP client (consume existing MCP servers)
- [ ] Tiered memory (working + episodic in SQLite)
- [ ] Basic policy engine (action → risk level → approval requirement)

Phase 2: Channels & Tools (Weeks 5-8)

Goal: Usable as a daily driver via Discord + CLI with real integrations.

- [ ] Discord channel adapter
- [ ] Built-in MCP servers: file system, shell, web fetch, calendar
- [ ] Composio integration for OAuth-managed tools
- [ ] Durable execution engine (Hatchet or Temporal)

- [ ] Confidence-based auto-approval
- [ ] Cost tracking and budget enforcement
- [ ] Semantic memory (vector embeddings)

Phase 3: Desktop & Polish (Weeks 9-12)

Goal: Desktop app with approval UI, multi-user support, observability.

- [ ] Tauri desktop app with approval queue UI
- [ ] Multi-user RBAC
- [ ] OpenTelemetry integration
- [ ] Webhook/API gateway for external triggers
- [ ] Advanced policy engine (trust tiers, time-based, budget-based)
- [ ] MCP server mode (expose assistant as a tool)
- [ ] Cron/scheduled workflows

Phase 4: Mobile & Enterprise (Weeks 13-20)

Goal: Mobile app, enterprise features, compliance readiness.

- [ ] Tauri mobile app (iOS/Android) with push notifications
- [ ] Slack/Teams channel adapters
- [ ] SOC2-ready audit trail export
- [ ] Multi-agent orchestration
- [ ] Skill/plugin marketplace
- [ ] Browser extension
- [ ] IDE plugin (VS Code)

---

9. Sources

Frameworks & Tools

- OpenClaw: https://github.com/openclaw/openclaw | https://en.wikipedia.org/wiki/OpenClaw
- LangGraph: https://www.langchain.com/langgraph
- CrewAI: https://crewai.com
- Microsoft AutoGen: https://github.com/microsoft/autogen | https://learn.microsoft.com/en-us/agent-framework/overview
- MCP: https://en.wikipedia.org/wiki/Model_Context_Protocol | https://www.thoughtworks.com/en-us/insights/blog/generative-ai/model-context-protocol-mcp-impact-2025
- Composio: https://composio.dev
- Arcade AI: https://www.arcade.dev
- Lindy.ai: https://www.lindy.ai | https://max-productive.ai/ai-tools/lindy/
- Dust.tt: https://dust.tt | https://www.cbinsights.com/company/dust
- Relevance AI: https://relevanceai.com/pricing
- n8n: https://n8n.io | https://n8n.io/pricing
- Taskade: https://www.taskade.com/pricing
- Devin: https://devin.ai | https://venturebeat.com/programming-development/devin-2-0-is-here
- Replit Agent: https://docs.replit.com/replitai/agent
- Cursor: https://www.cursor.com
- Windsurf: https://windsurf.com
- Aider: https://aider.chat
- Open Interpreter: https://www.openinterpreter.com | https://github.com/openinterpreter/open-interpreter
- GPT Pilot: https://github.com/Pythagora-io/gpt-pilot
- AutoGPT: https://github.com/Significant-Gravitas/AutoGPT
- Ruflo: https://github.com/ruvnet/ruflo

Orchestration & Infrastructure

- Temporal: https://temporal.io | https://temporal.io/blog/build-resilient-agentic-ai-with-temporal
- Inngest: https://www.inngest.com | https://www.inngest.com/blog/ai-orchestration-with-agentkit-step-ai
- Hatchet: https://hatchet.run | https://github.com/hatchet-dev/hatchet
- Prefect: https://www.prefect.io
- Dagster: https://dagster.io
- Inngest vs Temporal: https://akka.io/blog/inngest-vs-temporal

Research & Guides

- AI Agent Frameworks Comparison: https://www.langflow.org/blog/the-complete-guide-to-choosing-an-ai-agent-framework-in-2025
- Agent Frameworks Benchmark: https://langwatch.ai/blog/best-ai-agent-frameworks-in-2025
- Memory in AI Agents Survey: https://arxiv.org/abs/2512.13564
- Multi-Model Orchestration: https://arxiv.org/abs/2512.22402
- LLM Orchestration Frameworks: https://research.aimultiple.com/llm-orchestration/

- Agent Sandboxing (Google/K8s): https://opensource.googleblog.com/2025/11/unleashing-autonomous-ai-agents
- Agentic AI Security: https://arxiv.org/html/2510.23883v1
- HITL Patterns: https://zapier.com/blog/human-in-the-loop/ | https://www.permit.io/blog/human-in-the-loop-for-ai-agents-best-practices | https://learn.microsoft.com/en-us/agent-framework/workflows/human-in-the-loop
- MCP Security: https://www.descope.com/learn/post/mcp
- Tauri 2.0: https://v2.tauri.app | https://ainexislab.com/tauri-2-0-ai-app-desktop-development-techniques/

Report generated: March 3, 2026 Research depth: 19 frameworks surveyed, 11 challenge areas analyzed, 16 integration categories mapped, 6 orchestration engines compared